# CHAPTER 11
# Stream Processing

> A complex system that works is invariably found to have evolved from a simple system that works. The inverse proposition also appears to be true: A complex system designed from scratch never works and cannot be made to work.
>
> —John Gall, *Systemantics (1975)*

In Chapter 10 we discussed batch processing — techniques that read a set of files as input, and produce a new set of output files. The output is a form of *derived data*, that is, a dataset that can be re-created by running the batch process again if necessary. We saw how this simple but powerful idea can be used to create search indexes, recommendation systems, analytics and more.

However, one big assumption remained throughout Chapter 10: that the input is *bounded*, i.e. of a known and finite size, so the batch process knows when it has finished reading its input. For example, the sorting operation that is central to MapReduce must read its entire input before it can start producing output: it could happen that the very last input record is the one with the lowest key, and thus needs to be the very first output record, so starting the output early is not an option.

In reality, a lot of data is *unbounded* because it arrives gradually over time: your users produced data yesterday and today, and they will continue to produce more data tomorrow. Unless you go out of business, this process never ends, and so the data is never "complete" in any meaningful way [1]. Thus, batch processors must artificially divide the data into chunks of fixed duration: for example, processing a day's worth of data at the end of every day, or processing an hour's worth of data at the end of every hour.

The problem with daily batch processes is that changes in the input are only reflected in the output a day later, which is too slow for many impatient users. To reduce the

delay, we can run the processing more frequently — say, processing a second's worth of data at the end of every second — or even continuously, abandoning the fixed time-slices entirely, and simply processing every event as it happens. That is the idea behind *stream processing*.

In general, a stream refers to data that is incrementally made available over time. The concept appears in many places: in the `stdin` and `stdout` of Unix, programming languages (lazy lists) [2], filesystem APIs (such as Java's FileInputStream), TCP connections, delivering audio and video over the internet, and so on.

In this chapter we will look at *event streams* as a data management mechanism: the unbounded, incrementally-processed counterpart to the batch data we saw in the last chapter. We will first discuss how streams are represented, stored and transmitted over a network. In "Databases and streams" on page 436 we will investigate the relationship between streams and databases. And finally, in "Processing Streams" on page 448 we will explore approaches and tools for processing those streams continually, and ways how they can be used to build applications.

## Transmitting Event Streams

In the batch processing world, the inputs and outputs of a job are files (perhaps on a distributed filesystem). What does the streaming equivalent look like?

When the input is a file (a sequence of bytes), the first processing step is usually to parse it into a sequence of records. In a stream processing context, a record is more commonly known as an *event*, but it is essentially the same thing: a small, self-contained, immutable object containing the details of something that happened at some point in time. An event usually contains a timestamp indicating when it happened.

For example, the thing that happened might be an action that a user took, such as viewing a page or making a purchase. It might also originate from a machine, such as a periodic measurement from a temperature sensor, or a CPU utilization metric. In the example of "Batch Processing with Unix Tools" on page 379, each line of the web server log is an event.

An event may be encoded as a text string, or JSON, or perhaps some binary form, as discussed in Chapter 4. This allows you to store an event, for example by appending it to a file, inserting it into a relational table, or writing it to a document database. It also allows you to send the event over the network to another node in order to process it.

In batch processing, a file is written once and then potentially read by multiple jobs. Analogously, in streaming terminology, an event is generated once by a *producer* (also known as *publisher* or *sender*), and then potentially processed by multiple *con-*

*sumers* (*subscribers* or *recipients*). In a filesystem, a filename identifies a set of related records; in a streaming system, related events are usually grouped together into a *topic* or *stream*.

In principle, a file or database is sufficient to connect producers and consumers: a producer writes every event that it generates to the datastore, and each consumer periodically polls the datastore to check for events that appeared since it last ran. This is essentially what a batch process does when it processes a day's worth of data at the end of every day.

However, when moving towards continual processing with low delays, polling becomes expensive if the datastore is not designed for this kind of usage. The more often you poll, the lower the percentage of requests that return new events, and thus the higher the overheads become. Instead, it is better for consumers to be notified when new events appear.

Databases have traditionally not supported this kind of notification mechanism very well: relational databases commonly have *triggers*, which can react to a change (e.g. a row being inserted into a table), but they are very limited in what they can do, and have been somewhat of an afterthought in database design [3, 4]. Instead, specialized tools were developed for the purpose of delivering event notifications.

## Messaging systems

A common approach for notifying consumers about new events is to use a *messaging system*: a producer sends a message containing the event, which is then pushed to consumers. We touched on these systems previously in "Message passing data flow" on page 132, but we will now go into more detail.

A direct communication channel like a Unix pipe or TCP connection between producer and consumer would be a simple way of implementing a messaging system. However, most messaging systems expand on this basic model. In particular, Unix pipes and TCP connect exactly one sender with one recipient, whereas a messaging system allows multiple producer nodes to send messages to the same topic, and allows multiple consumer nodes to receive messages in a topic.

Within this *publish-subscribe* model, different systems take a wide range of approaches, and there is no one right answer for all purposes. To differentiate the systems, it is particularly helpful to ask the following two questions:

1. *What happens if the producers send messages faster than the consumers can process them?* Broadly speaking, there are three options: the system can drop messages, buffer messages in a queue, or apply *backpressure* (block the producer from sending more messages). For example, Unix pipes and TCP use backpres-

sure: they have a small fixed-size buffer, and if it fills up, the sender is blocked until the recipient takes data out of the buffer.

If messages are buffered in a queue, it is important to understand what happens as that queue grows. Does the system crash if the queue no longer fits in memory? Or does it write messages to disk, but how does that affect performance?

2. *What happens if nodes crash or temporarily go offline? Are any messages lost?* As with databases, durability may require some combination of writing to disk and/or replication (see "Replication and durability" on page 218), which has a cost. If you can afford to sometimes lose messages, you can probably get higher throughput and lower latency on the same hardware.

Whether message loss is acceptable depends very much on the application. For example, with sensor readings and metrics that are transmitted periodically, an occasional missing data point is perhaps not important, since an updated value will be sent a short time later anyway. However, beware that if a large number of messages is dropped, it may not be immediately apparent that the metrics are incorrect [5]. If you are counting events, it is more important that they are delivered reliably, since every lost message means incorrect counters.

A nice property of the batch processing systems in Chapter 10 is that they provide a strong reliability guarantee: failed tasks are automatically retried, and partial output from failed tasks is automatically discarded. This means the output is the same as if no failures had occurred, which helps simplify the programming model. Later in this chapter we will examine how we can provide similar guarantees in a streaming context.

#### Direct messaging from producers to consumers

A number of messaging systems use direct network communication between producers and consumers, without going via intermediary nodes:

- UDP multicast is widely used in the financial industry for streams such as stock market feeds, where low latency is important [6]. Although UDP itself is unreliable, application-level protocols can recover lost packets (the producer must remember packets it has sent, so that it can retransmit them on demand).
- Brokerless messaging libraries such as ZeroMQ [7] and nanomsg take a similar approach, implementing publish-subscribe messaging over TCP or IP multicast.
- StatsD [8] and Brubeck [5] use unreliable UDP messaging for collecting metrics from all machines on the network, and monitoring them. (In the StatsD protocol, counter metrics are only correct if all messages are received; using UDP makes the metrics at best approximate [9]. See also "TCP versus UDP" on page 275.)

- If the consumer exposes a service on the network, producers can make a direct HTTP or RPC request (see "Data flow through services: REST and RPC" on page 127) to push messages to the consumer. This is the idea behind webhooks [10], a pattern in which a callback URL of one service is registered with another service, and it makes a request to that URL whenever an event occurs.

Although these direct messaging systems work well in the situations for which they are designed, they generally require the application code to be aware of the possibility of message loss. The faults they can tolerate are quite limited: even if the protocols detect and retransmit packets that are lost in the network, they generally assume that producers and consumers are constantly online.

If a consumer is offline, it may miss messages that were sent while it is unreachable. Some protocols allow the producer to retry failed message deliveries, but this approach may break down if the producer crashes, losing the buffer of messages that it was supposed to retry.

### Message brokers

A widely-used alternative is to send messages via a *message broker* (also known as *message queue*), which is essentially a kind of database that is optimized for handling message streams [11]. It runs as a server, with producers and consumers connecting to it as clients. Producers write messages to the broker, and consumers receive them by reading them from the broker.

By centralizing the data in the broker, these systems can more easily tolerate clients that come and go (connect, disconnect and crash), and the question of durability is moved to the broker instead. Some message brokers only keep messages in memory, while others (depending on configuration) write them to disk so that they are not lost in case of a broker crash. Faced with slow consumers, they generally allow unbounded queueing (as opposed to dropping messages or backpressure), although this may also depend on the configuration.

A consequence of queueing is also that consumers are generally *asynchronous*: when a producer sends a message, it normally only waits for the broker to confirm that it has buffered the message, but it does not wait for the message to be processed by consumers. The delivery to consumers will happen at some undetermined future point in time — often within a fraction of a second, but sometimes significantly later if there is a queue backlog.

### Message brokers compared to databases

Some message brokers can even participate in 2-phase commit protocols using XA or JTA (see "Distributed transactions in practice" on page 350). This makes them quite

similar in nature to databases, although there are still important practical differences between message brokers and databases:

- Databases usually keep data forever until it is explicitly deleted, whereas most message brokers automatically delete a message when it has been successfully delivered to its consumers. Such message brokers are not suitable for long-term data storage.
- Since they quickly delete messages, most message brokers assume that their working set is fairly small, i.e. the queues are short. If the broker needs to buffer a lot of messages because the consumers are slow (perhaps spilling messages to disk if they no longer fit in memory), each individual message takes longer to process, and the overall throughput may degrade [12].
- Databases often support secondary indexes and various ways of searching for data, while message brokers often support some way of subscribing to a subset of topics matching some pattern. The mechanisms are different, but both are essentially ways for a client to select the portion of the data that it wants to know about.
- When querying a database, the result is typically based on a point-in-time snapshot of the data; if another client subsequently writes something to the database that changes the query result, the first client does not find out that its prior result is now outdated (unless it repeats the query, i.e. polls for changes). By contrast, message brokers do not support arbitrary queries, but they do notify clients when data changes (i.e. when new messages become available).

This is the traditional view of message brokers, which is encapsulated in standards like JMS [13] and AMQP [14], and implemented in software like RabbitMQ, ActiveMQ, HornetQ, Qpid, TIBCO Enterprise Message Service, IBM MQ, and Azure Service Bus.

### Multiple consumers

When multiple consumers are reading messages in the same topic, two main patterns of messaging are used, as illustrated in Figure 11-1:

*Load balancing*
: Each message is delivered to *one* of the consumers, so the consumers can share the work of processing the messages in the topic. The broker may assign messages to consumers arbitrarily. This is useful when the messages are expensive to process, and so you want to be able to add consumers to parallelize the processing. (In AMQP, this is done by having multiple clients consuming from the same queue, and in JMS it is called a *shared subscription*.)

*Fan-out*

: Each message is delivered to *all* of the consumers. This allows several independent consumers to each "tune in" to the same broadcast of messages, without affecting each other — the streaming equivalent of having several different batch jobs that read the same input file. (This is done with topic subscriptions in JMS, and exchange bindings in AMQP.)

The two patterns can be combined: for example, two separate groups of consumers may each subscribe to a topic, such that each group collectively receives all messages, but within each group only one of the nodes receives each message.

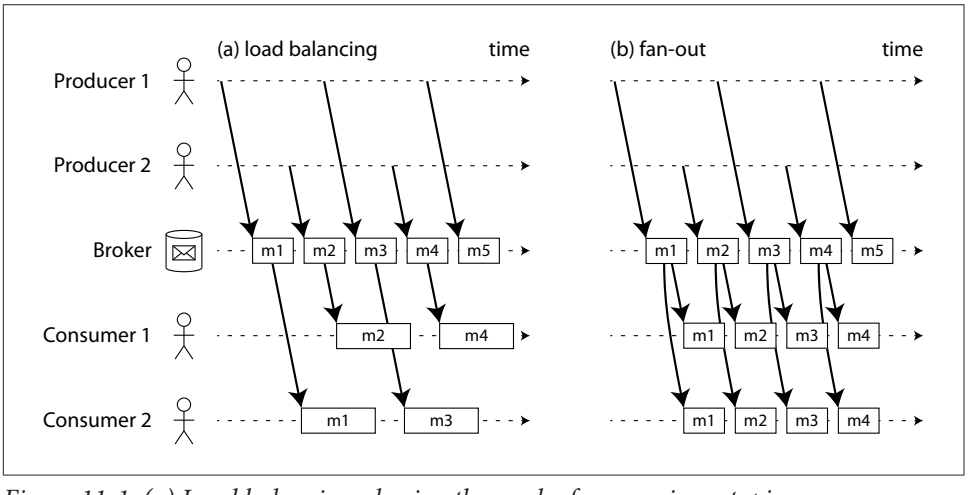


*Figure 11-1. (a) Load balancing: sharing the work of consuming a topic among consumers; (b) fan-out: delivering each message to multiple consumers.*

### Acknowledgements and redelivery

Consumers may crash at any time, so it could happen that a broker delivers a message to a consumer, but the consumer never processes it, or only partially processes it before crashing. In order to ensure that the message is not lost, message brokers use *acknowledgements*: a client must explicitly tell the broker when it has finished processing a message, so that the broker can remove it from the queue.

If the connection to a client is closed or times out without the broker receiving an acknowledgement, it assumes that the message was not processed, and therefore it delivers the message again to another consumer. (Note that it could happen that the message actually *was* fully processed, but the acknowledgement was lost in the network. To handle this case requires an atomic commit protocol, as discussed in "Distributed transactions in practice" on page 350.)

When combined with load balancing, this redelivery behavior has an interesting effect on the ordering of messages. In Figure 11-2, the consumers generally process

messages in the order they were sent by producers. However, consumer 2 crashes while processing message m3, at the same time as consumer 1 is processing message m4. The unacknowledged message m3 is subsequently redelivered to consumer 1, with the result that consumer 1 processes messages in the order m4, m3, m5. Thus, m3 and m4 are not delivered in the same order as they were sent by producer 1.

Even if the message broker otherwise tries to preserve the order of messages (as required by both the JMS and AMQP standards), the combination of load balancing with redelivery inevitably leads to messages being reordered. This is not a problem if messages are completely independent of each other, but it can be important if there are causal dependencies between messages, as we shall see later in the chapter.

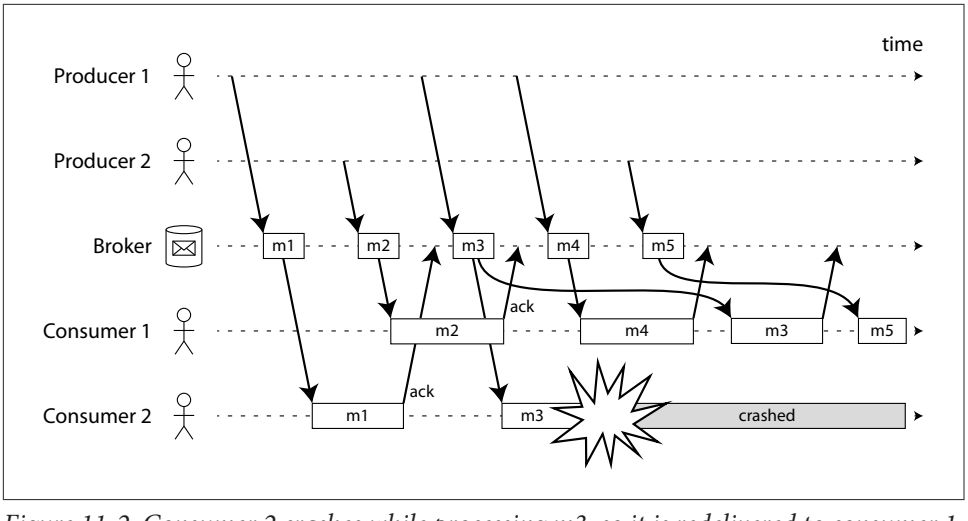


*Figure 11-2. Consumer 2 crashes while processing m3, so it is redelivered to consumer 1 at a later time.*

## Partitioned logs

Sending a packet over a network, or making a request to a network service, is normally a transient operation that leaves no permanent trace. Although it is possible to record it permanently (using packet capture and logging), we normally don't think of it that way. Even message brokers that durably write messages to disk quickly delete them again after they have been delivered to consumers, because they are built around a transient messaging mindset.

Databases and filesystems take the opposite approach: everything that is written to a database or file is normally expected to be permanently recorded, at least until someone explicitly chooses to delete it again.

This difference in mindset has a big impact on how derived data is created. A key feature of batch processes, as discussed in Chapter 10, is that you can run them

repeatedly, experimenting with the processing steps, without risk of damaging the input (since the input is read-only). This is not the case with AMQP/JMS-style messaging: receiving a message is destructive if processing it causes it to be deleted from the broker, so you cannot run the same consumer again and expect to get the same result.

If you add a new consumer to a messaging system, it typically only starts receiving messages from the time it was registered, but no prior messages (since they are already gone). Contrast this with files and databases, where you can add a new client at any time, and it can read data written arbitrarily far in the past (as long as it has not been overwritten or deleted).

Why can we not have a hybrid, combining the durable storage approach of databases with the low-latency notification facilities of messaging? This is the idea behind *log-based message brokers*.

#### Using logs for message storage

A log is simply an append-only sequence of records on disk. We previously discussed logs in the context of log-structured storage engines and write-ahead logs in Chapter 3, and in the context of replication in "Implementation of replication logs" on page 152.

The same structure can be used to implement a message broker: a producer sends a message by appending it to the end of the log, and a consumer receives messages by reading the log sequentially. If a consumer reaches the end of the log, it waits for a notification that a new message has been appended. The Unix tool `tail -f`, which watches a file for data being appended, essentially works like this.

In order to scale to higher throughput than a single disk can offer, the log can be *partitioned* (in the sense of Chapter 6). Different partitions can then be hosted on different machines, and several partitions can be grouped together to a topic. This approach is illustrated in Figure 11-3.

Within each partition, the broker assigns a monotonically increasing sequence number, or *offset*, to every message (in Figure 11-3, the numbers in boxes are message offsets). Such a sequence number makes sense because a partition is append-only, so the messages within a partition are totally ordered. There is no ordering guarantee across different partitions.

Apache Kafka [15, 16], Amazon Kinesis Streams [17] and Twitter's DistributedLog [18, 19] are log-based message brokers that work like this. Even though they write all messages to disk, they able to achieve throughput of millions of messages per second by partitioning across multiple machines, and fault tolerance by replicating messages [20, 21].

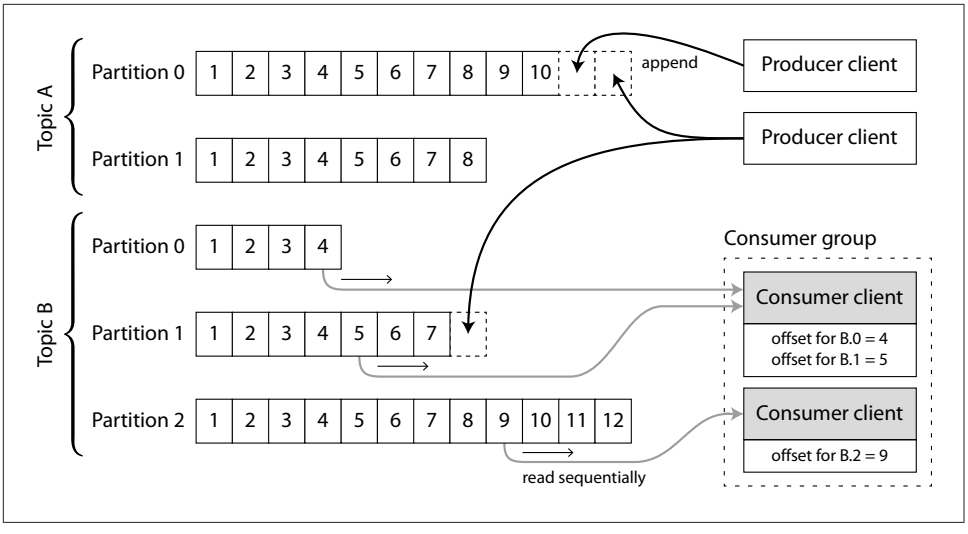


*Figure 11-3. Producers send messages by appending them to a topic-partition file, and consumers read these files sequentially.*

### Logs compared to traditional messaging

The log-based approach trivially supports fan-out messaging, because several consumers can independently read the log without affecting each other. To achieve load balancing across a group of consumers, instead of assigning individual messages to consumer clients, the broker can assign entire partitions to nodes in the consumer group.

Each client then consumes *all* the messages in the partitions it has been assigned. Typically, when a consumer has been assigned a log partition, it reads the messages in the partition sequentially, in a straightforward single-threaded manner. This coarse-grained load balancing approach has some downsides:

- The number of nodes sharing the work of consuming a topic can be at most the number of log partitions in that topic, because messages within the same partition are delivered to the same node.[i]

i. It's possible to create a load balancing scheme in which two consumers share the work of processing a partition by having both read the full set of messages, but one only processes messages with even-numbered offsets while the other processes odd-numbered offsets. Or you could spread message processing over a thread pool, but that approach complicates consumer offset management. In general, single-threaded processing of a partition is preferable.

- If a single message is slow to process, it holds up the processing of subsequent messages in that partition (a form of head-of-line blocking, see "Describing performance" on page 11).

Thus, in situations where messages may be expensive to process and you want to parallelize processing on a message-by-message basis, and where message ordering is not so important, the JMS/AMQP style of message broker is preferable. On the other hand, in situations with high message throughput, where each message is fast to process and where message ordering is important, the log-based approach works very well.

#### Consumer offsets

Consuming a partition sequentially makes it easy to tell which messages have been processed: all messages with an offset less than a consumer's current offset have already been processed, and all messages with a greater offset have not yet been seen. Thus, the broker does not need to track acknowledgements for every single message — it only needs to periodically record the consumer offsets. The reduced bookkeeping overhead, and the opportunities for batching and pipelining in this approach, help increase the throughput of the system.

This offset is in fact very similar to the *log sequence number* that is commonly found in single-leader database replication, and which we discussed in "Setting up new followers" on page 149. In database replication, the log sequence number allows a follower to reconnect to a leader after it has become disconnected, and resume replication without skipping any writes. Exactly the same principle is used here: the message broker behaves like a leader database, and the consumer like a follower.

If a consumer node fails, another node in the consumer group is assigned the failed consumer's partitions, and it starts consuming messages at the last recorded offset. If the consumer had processed subsequent messages, but not yet recorded their offset, those messages will be processed a second time on restart. We will discuss ways of dealing with this issue later in the chapter.

#### Disk space usage

If you only ever append to the log, you will eventually run out of disk space. To reclaim disk space, the log is actually divided into segments, and from time to time old segments are deleted or moved to archive storage. (We discuss a more sophisticated way of freeing disk space later.)

This means that if a slow consumer cannot keep up with the rate of messages, and it falls so far behind that its consumer offset points to a deleted segment, it will miss some of the messages. Effectively, the log implements a bounded-size buffer that dis-

cards old messages when it gets full, also known as a *circular buffer* or *ring buffer*. However, since that buffer is on disk, it can be quite large.

Let's do a back-of-the-envelope calculation. At the time of writing, a typical large hard drive has a capacity of 6 TB and a sequential write throughput of 150 MB/s. If you are writing messages at the fastest possible rate, it takes about 11 hours to fill the drive. Thus, the disk can buffer 11 hours worth of messages, after which it will start overwriting old messages. This ratio remains the same, even if you use many hard drives and machines. In practice, deployments rarely use the full write bandwidth of the disk, so the log can typically keep a buffer of several days or even weeks worth of messages.

You can monitor how far a consumer is behind the head of the log, and raise an alert if it falls behind. As the buffer is large, there is enough time for a human to fix the slow consumer and allow it to catch up before it starts missing messages. And even if the consumer does fall too far behind, it only affects itself, but it does not disrupt the service for other consumers, which is a big operational advantage.

#### Replaying old messages

We noted previously that with AMQP and JMS-style message brokers, processing and acknowledging messages is a destructive operation, since it causes the messages to be deleted on the broker. On the other hand, in a log-based message broker, consuming messages is more like reading from a file: it is a read-only operation that does not change the log.

The only side-effect of processing, besides any output of the consumer, is that the consumer offset moves forward. But the offset is under the consumer's control, so it can easily be manipulated if necessary: for example, you can start a copy of a consumer with yesterday's offsets, and write the output to a different location, in order to re-process the last day's worth of messages. You can repeat this any number of times, varying the processing code.

This aspect makes log-based messaging more like the batch processes of the last chapter, where derived data is clearly separated from input data through a repeatable transformation process. It allows more experimentation and easier recovery from errors and bugs, making it a good tool for integrating data flows within an organization [22].

## Databases and streams

We have drawn some comparisons between message brokers and databases. Even though they have traditionally been considered to be separate categories of tools, we saw that log-based message brokers have been successful in taking ideas from data-

bases and applying them to messaging. We can also go in reverse: take ideas from messaging and streams, and apply them to databases.

We said previously that an event is a record of something that happened at some point in time. The thing that happened may be a user action (e.g. typing a search query), or a sensor reading, but it may also be a *write to a database*. The fact that something was written to a database is an event that can be captured, stored and processed. This suggests that the connection between databases and streams runs deeper than just the physical storage of logs on disk — it is quite fundamental.

In fact, a replication log (see "Implementation of replication logs" on page 152) is a stream of database write events, produced by the leader as it processes transactions. The followers apply that stream of writes to their own copy of the database, and thus end up with an up-to-date copy of the same data. The events in the replication log describe the data changes that occurred.

We also came across the *state machine replication* principle in "Total order broadcast" on page 338, which states: if every event represents a write to the database, and every replica processes the same events in the same order, then the replicas will all end up in the same final state. (Processing an event is assumed to be a deterministic operation.) It's just another case of event streams!

In this section we will first look at a problem that arises in heterogeneous data systems, and then explore how we can solve it by bringing ideas from event streams to databases.

## Keeping systems in sync

As we have seen throughout this book, there is no single system that can satisfy all data storage, querying and processing needs. In practice, most non-trivial web applications need to combine several different technologies in order to satisfy their requirements: for example, using an OLTP database to serve user requests, a cache to speed up common requests, a full-text index to handle search queries, and a data warehouse for analytics. Each of these has its own copy of the data, stored in its own representation that is optimized for its purposes.

As the same data appears in all these different places, they need to be kept in sync with each other: if an item is updated in the database, it also needs to be updated in the cache, search indexes and the data warehouse. With data warehouses, this is usually done through ETL processes (see "Data warehousing" on page 88) — often by taking a full copy of a database, transforming it, and bulk-loading it into the data warehouse. In other words, a batch process.

If periodic full database dumps are too slow, an alternative that is sometimes used is *dual writes*, in which the application code explicitly writes to each of the systems

when data changes: for example, first writing to the database, then updating the search index, then invalidating the cache entries.

However, dual writes have some serious problems, one of which is a race condition illustrated in Figure 11-4. In this example, two clients concurrently want to update an item X: client 1 wants to set the value to A, and client 2 wants to set it to B. Both clients first write the new value to the database, then write it to the search index.

Due to unlucky timing, the requests are interleaved: the database first sees the write from client 1 setting the value to A, then the write from client 2 setting the value to B, so the final value in the database is B. The search index first sees the write from client 2, then client 1, so the final value in the search index is A. The two systems are now permanently inconsistent with each other, even though no error occurred in the execution.

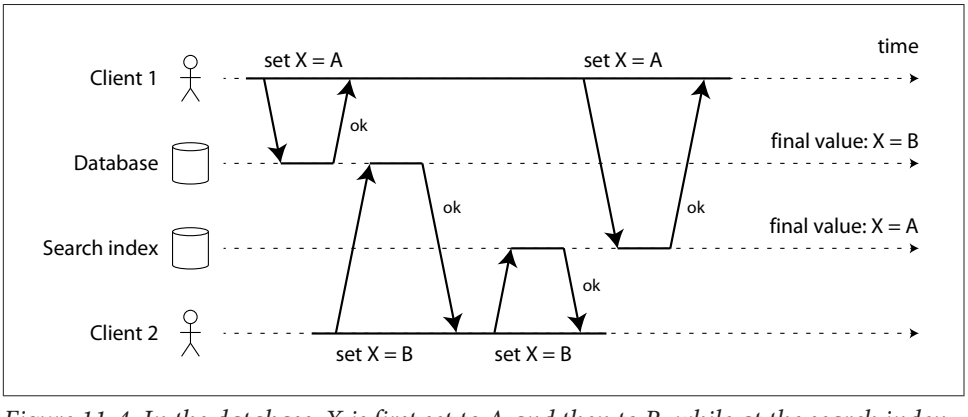


*Figure 11-4. In the database, X is first set to A and then to B, while at the search index the writes arrive in the opposite order.*

Unless you have some additional concurrency tracking mechanism, such as the version vectors we discussed in "Detecting concurrent writes" on page 178, you will not even notice that concurrent writes occurred — one value will simply silently overwrite another value.

Another problem with dual writes is that one of the writes may fail while the other succeeds. This is a fault-tolerance problem rather than a concurrency problem, but it also has the effect of the two systems becoming inconsistent with each other. Ensuring that they either both succeed or both fail is a case of the atomic commit problem, which is expensive to solve (see "Atomic commit and two-phase commit (2PC)" on page 344).

If you only have one replicated database with a single leader, then that leader determines the order of writes, so the state machine replication approach works among replicas of the database. However, in Figure 11-4 there isn't a single leader: the data-

base may have a leader and the search index may have a leader, but neither follows the other, and so conflicts can occur (see "Multi-leader replication" on page 161).

The situation would be better if there really was only one leader, for example the database, and if we could make the search index a follower of the database. But is this possible in practice?

## Change data capture

The problem with most databases' replication logs is that they have long been considered to be an internal implementation detail of the database, not a public API. Clients are supposed to query the database through its data model and query language, not parse the replication logs and try to extract data from them.

For decades, many databases simply did not have a documented way of getting the log of changes written to it. For this reason it was difficult to take all the changes made in a database and replicate them to a different storage technology such as a search index, cache or data warehouse.

More recently, there has been growing interest in *change data capture* (CDC), which is the process of observing all data changes written to a database, and extracting them in a form in which they can be replicated to other systems. CDC is especially interesting if changes are made available as a stream, immediately as they are written.

For example, you can capture the changes in a database and continually apply the same changes to a search index. If the log of changes is applied in the same order, you can expect the data in the search index to match the data in the database. The search index and any other derived data systems are just consumers of the change stream, as illustrated in Figure 11-5.

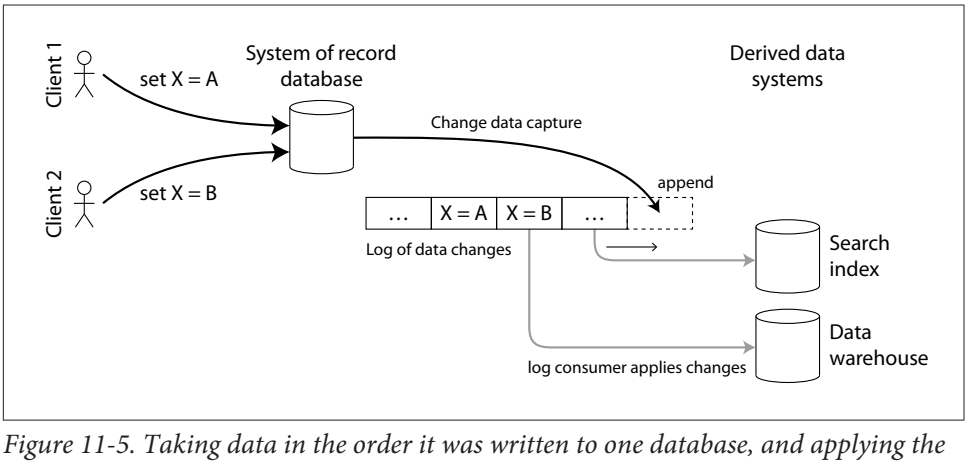


*Figure 11-5. Taking data in the order it was written to one database, and applying the changes to other systems in the same order.*

### Implementing change data capture

We can call the log consumers *derived data systems*, as discussed in the introduction to Part III: the data stored in the search index and the data warehouse is just another view onto the data in the system of record. Change data capture is a mechanism for ensuring that all changes made to the system of record are also reflected in the derived data systems, so that the derived systems have an accurate copy of the data.

Essentially, change data capture makes one database the leader (the one from which the changes are captured), and turns the others into followers. A log-based message broker is well suited for transporting the change events from the source database, since it preserves the ordering of messages (avoiding the reordering issue of Figure 11-2).

Database triggers can be used to implement change data capture (see "Trigger-based replication" on page 154), by registering triggers that observe all changes to data tables and add an entry to a changelog table. However, they tend to be fragile and have significant performance overheads. Parsing the replication log can be a more robust approach, although it also comes with challenges, such as handling schema changes.

LinkedIn's Databus [23], Facebook's Wormhole [24] and Yahoo's Sherpa [25] use this idea at large scale. Bottled Water implements CDC for PostgreSQL using an API that decodes the write-ahead log [26], Maxwell and Debezium do something similar for MySQL by parsing the binlog [27, 28], Mongoriver reads the MongoDB oplog [29, 30], and GoldenGate provides similar facilities for Oracle [31, 32].

Like message brokers, change data capture is usually asynchronous: the system of record database does not wait for the change to be applied to consumers before committing it. This has the operational advantage that adding a slow consumer does not slow down the system of record, but it has the downside that all the issues of replication lag apply (see "Problems With Replication Lag" on page 155).

### Initial snapshot

If you have the log of all changes that were ever made to a database, you can reconstruct the entire state of the database by replaying the log. However, in many cases, keeping all changes forever would require too much disk space, and replaying it would take too long, so the log needs to be truncated.

Building a new full-text index, for example, requires a full copy of the entire database — it is not sufficient to only apply a log of recent changes, since it would be missing items that were not recently updated. Thus, if you don't have the entire log history, you need to start with a consistent snapshot, as previously discussed in "Setting up new followers" on page 149.

The snapshot of the database must correspond to a known position or offset in the change log, so that you know at which point to start applying changes after the snapshot has been processed. Some CDC tools integrate this snapshot facility, while others leave it as a manual operation.

#### Log compaction

If you can only keep a limited amount of log history, you need to go through the snapshot process every time you want to add a new derived data system. However, *log compaction* provides a good alternative.

We discussed log compaction previously in "Hash indexes" on page 70, in the context of log-structured storage engines (see Figure 3-2 for an example). The principle is simple: the storage engine periodically looks for log records with the same key, throws away any duplicates and keeps only the most recent update for each key. This compaction and merging process runs in the background.

An update with a special null value indicates that a key was deleted. But as long as a key is not overwritten or deleted, it stays in the log forever. The disk space required for such a compacted log depends only on the current contents of the database, not the number of writes that have ever occurred in the database. If the same key is frequently overwritten, previous values will eventually be garbage-collected, and only the latest value will be retained.

The same idea works in the context of log-based message brokers and change data capture. If the CDC system is set up such that every change has a primary key, and every update for a key replaces the previous value for that key, then it's sufficient to keep just the most recent write for a particular key.

Now, whenever you want to rebuild a derived data system such as a search index, you can start a new consumer from offset 0 of the log-compacted topic, and sequentially scan over all messages in the log. The log is guaranteed to contain the most recent value for every key in the database (and maybe some older values) — in other words, it can obtain a full copy of the database contents without having to take another snapshot on the CDC source database.

This log compaction feature is supported by Apache Kafka. As we shall see later in this chapter, it allows the message broker to be used for durable storage, not just for transient messaging.

#### API support for change streams

Increasingly, databases are beginning to support change streams as a first-class interface, rather than the retrofitted and reverse-engineered efforts that change data capture has long been. For example, RethinkDB allows queries to subscribe to notifications when the results of the query change [33], Firebase [34] and CouchDB

[35] provide data synchronization based on a change feed that is also made available to applications, and Meteor uses the MongoDB oplog to subscribe to data changes and update the user interface [36].

Kafka Connect [37] is an effort to integrate change data capture tools for a wide range of database systems with Kafka. Once the stream of change events is in Kafka, it can be used to update derived data systems such as search indexes, and also feed into stream processing systems as discussed later in this chapter.

## Event sourcing

There are some parallels between the ideas we've discussed here and *event sourcing*, a technique that was developed in the Domain-Driven Design (DDD) community [38, 39, 40]. We will discuss event sourcing briefly, because it incorporates some useful and relevant ideas for streaming systems.

Similarly to change data capture, event sourcing involves storing all changes to the application state as a log of change events. The biggest difference is that event sourcing applies the idea at a different level of abstraction:

- In change data capture, the application uses the database in a mutable way, updating and deleting records at will. The log of changes is extracted from the database at a low level (e.g. by parsing the replication log), which ensures that the order of writes extracted from the database matches the order in which they were actually written, avoiding the race condition in Figure 11-4. The application writing to the database does not need to be aware that CDC is occurring.
- In event sourcing, the application logic is explicitly built on the basis of immutable events that are written to an event log. In this case, the event store is append-only, and updates or deletes are discouraged or prohibited. Events are carefully designed to mirror things that happened at the application level.

Event sourcing is similar to the chronicle data model [41], and there are also similarities between an event log and the fact table that you find in a star schema (see "Stars and snowflakes: schemas for analytics" on page 90).

Specialized databases such as Event Store [42] have been developed to support applications using event sourcing, but in general the approach is independent of any particular tool. A conventional database or a log-based message broker can also be used to build applications in this style.

### Deriving current state from the event log

An event log by itself is not very useful, because users generally expect to see the current state of the system, not the history of modifications. For example, on a shopping

website, users expect to be able to see the current contents of their cart, not an append-only list of all the changes they have ever made to their cart.

Thus, applications that use event sourcing need to take the log of events (representing the data *written* to the system) and transform it into application state that is suitable for showing to user (the way in which data is *read* from the system [43]). This transformation can use arbitrary logic, but it should be deterministic, so that you can run it again and derive the same application state from the event log.

Like with change data capture, replaying the event log allows you to reconstruct the current state of the system. However, log compaction needs to be handled differently:

- A CDC event for the update of a record typically contains the entire new version of the record, so the current value for a primary key is entirely determined by the most recent event for that primary key, and log compaction can discard previous events for the same key.
- On the other hand, with event sourcing, events are modeled at a higher level: an event typically expresses the intent of a user action, not the mechanics of the state update that occurred as a result of the action. In this case, later events typically do not override prior events, and so you need the full history of events to reconstruct the final state. Log compaction is not possible in the same way.

Applications that use event sourcing typically have some mechanism for storing snapshots of the current state that is derived from the log of events, so they don't need to repeatedly re-process the full log. However, this is only a performance optimization to speed up reads and recovery from crashes; the intention is that the system is able to store all raw events forever, and re-process the full event log whenever required. This is a reasonable assumption for all but the very largest applications.

#### Commands and events

The event sourcing philosophy is careful to distinguish between *events* and *commands* [44]. When a request from a user first arrives, it is initially a command: at this point it may still fail, for example because some integrity condition is violated. The application must first validate that it can execute the command. If the validation is successful and the command is accepted, it becomes an event, which is durable and immutable.

For example, if a user tries to register a particular username, or reserve a seat on an airplane or in a theater, then the application needs to check that the username or seat is not already taken. (We previously discussed this example in "Fault-tolerant consensus" on page 355.) When that check has succeeded, the application can generate an event to indicate that a particular username was registered by a particular user ID, or that a particular seat has been reserved for a particular customer.

At the point when the event is generated, it becomes a *fact*. Even if the customer later decides to change or cancel the reservation, the fact remains true that they formerly held a reservation for a particular seat, and the change or cancellation is a separate event that is added later.

A consumer of the event stream is not allowed to reject an event: by the time the consumer sees the event, it is already an immutable part of the log, and it may have already been seen by other consumers. Thus, any validation of a command needs to happen synchronously, before it becomes an event.

Alternatively, the user request to reserve a seat could be split into two events: first a tentative reservation, and then a separate confirmation event once the reservation has been validated (as discussed in "Implementing linearizable storage using total order broadcast" on page 340). This allows the validation to take place in an asynchronous process.

## State, streams, and immutability

We saw in Chapter 10 that batch processing benefits from the immutability of its input files, so you can run experimental processing jobs on existing input files without fear of damaging them. This principle of immutability is also what makes event sourcing and change data capture so powerful.

We normally think of databases as storing the current state of the application — this representation is optimized for reads, and it is usually the most convenient for serving queries. The nature of state is that it changes, which is why databases support updating and deleting data as well as inserting it. How does this fit with immutability?

Whenever you have state that changes, that state is the result of the events that mutated it over time. For example, your list of currently available seats is the result of the reservations you have processed, the current account balance is the result of the credits and debits on the account, and the response time graph for your web server is an aggregation of the individual response times of all web requests that occurred.

No matter how the state changes, there was always a sequence of events that caused those changes. Even as things are done and undone, the fact remains true that those events occurred. The important thing to realize is that mutable state and an append-only log of immutable events do not contradict each other: they are two sides of the same coin. The log of all changes, the *changelog*, represents the evolution of state over time.

If you are mathematically inclined, you might say that the application state is what you get when you integrate an event stream over time, and a change stream is what you get when you differentiate the state by time, as shown in Figure 11-6 [45, 46].

The analogy has limitations (for example, the second derivative of state does not seem to be meaningful), but it's a useful starting point for thinking about data.

$$state(now) = \int_{t=0}^{now} stream(t)\ \mathrm{d}t \qquad stream(t) = \frac{\mathrm{d}\ state(t)}{\mathrm{d}t}$$

*Figure 11-6. The relationship between the current application state and an event stream.*

If you store the changelog durably, that simply has the effect of making the state reproducible. If you consider the log of events to be your system of record, and any mutable state as being derived from it, it becomes easier to reason about the flow of data through a system. As Pat Helland puts it [47]:

> Transaction logs record all the changes made to the database. High-speed appends are the only way to change the log. From this perspective, the contents of the database hold a caching of the latest record values in the logs. The truth is the log. The database is a cache of a subset of the log. That cached subset happens to be the latest value of each record and index value from the log.

### Advantages of immutable events

Immutability in databases is an old idea. For example, accountants have been using immutability for centuries in financial bookkeeping. When a transaction occurs, it is recorded in an append-only *ledger*, which is essentially a log of events describing money, goods or services that have changed hands. The accounts, such as profit and loss or the balance sheet, are derived from the transactions in the ledger by adding them up [48].

If a mistake is made, accountants don't erase or change the incorrect transaction in the ledger — instead, they add another transaction that compensates for the mistake, for example refunding an incorrect charge. The incorrect transaction still remains in the ledger forever, because it might be important for auditing reasons. If incorrect figures, derived from the incorrect ledger, have already been published, then the figures for the next accounting period include a correction. This process is entirely normal in accounting [49].

Although such auditability is particularly important in financial systems, it is also beneficial for many other systems that are not subject to such strict regulation. As discussed in "Philosophy of batch process outputs" on page 401, if you accidentally deploy buggy code that writes bad data to a database, recovery is much harder if the

code is able to destructively overwrite data. With an append-only log of immutable events, it is much easier to see what happened and recover.

Immutable events also capture more information than just the current state. For example, on a shopping website, a customer may add an item to their cart and then remove it again. Although the second event cancels out the first event from the point of view of order fulfillment, it may be useful to know for analytics purposes that the customer was considering a particular item but then decided against it. Perhaps they will choose to buy it in future, or perhaps they found a substitute. This information is recorded in an event log, but would be lost in a database that deletes items when they are removed from the cart [38].

#### Deriving several views from the same event log

Moreover, by separating mutable state from the immutable event log, you can derive several different read-oriented representations from the same log of events. This works just like having multiple consumers of a stream (Figure 11-5).

Having an explicit translation step from an event log to a database makes it easier to evolve your application over time: if you want to introduce a new feature that presents your existing data in some new way, you can use the event log to build a separate read-optimized view for the new feature, and run it alongside the existing systems without having to modify them. Running old and new systems side-by-side is often easier than performing a complicated schema migration in an existing system. Once the old system is no longer needed, you can simply shut it down [43, 50].

Storing data is normally quite straightforward if you don't have to worry about how it is going to be queried and accessed; a lot of the complexities of schema design, indexing and storage engines are the result of wanting to support certain query and access patterns (see Chapter 3). For this reason, you gain a lot of flexibility by separating the form in which data is written from the form it is read, and by allowing several different read views. This idea is sometimes known as *command query responsibility segregation* or CQRS [38, 51, 52].

The traditional approach to database and schema design is based on the fallacy that data must be written in the same form as it will be queried. Debates about normalization and denormalization (see "Many-to-one and many-to-many relationships" on page 31) become largely irrelevant if you can translate data from a write-optimized event log to read-optimized application state: it is entirely reasonable to denormalize data in the read-optimized views, as the translation process gives you a mechanism for keeping it up-to-date.

In "Describing load" on page 9 we discussed Twitter's home timelines, a cache of recently-written tweets by the people a particular user is following (like a mailbox). This is another example of read-optimized state: home timelines are highly denormalized, since your tweets are duplicated in all of the timelines of the people follow-

ing you. However, the fan-out service keeps this duplicated state in sync with new tweets and new following relationships, which keeps the duplication manageable.

### Concurrency control

The biggest downside of event sourcing and change data capture is that the consumers of the event log are usually asynchronous, so there is a possibility that a user may make a write to the log, then read from a log-derived view, and find that their write has not yet been reflected in the read view. We discussed this problem and potential solutions previously in "Reading your own writes" on page 156.

One solution would be to perform the updates of the read view synchronously with appending the event to the log. This requires a transaction to combine the writes into an atomic unit, so you either need to keep the event log and the read view in the same storage system, or you need a distributed transaction across the different systems. Alternatively, you could use the approach discussed in "Implementing linearizable storage using total order broadcast" on page 340.

On the other hand, deriving the current state from an event log also simplifies some aspects of concurrency control. Much of the need for multi-object transactions (see "Single-object and multi-object operations" on page 219) stems from a single user action requiring data to be changed in several different places. With event sourcing, you can design an event such that it is a self-contained description of a user action — so the user action requires only a single write in one place, namely appending the event to the log, which is easy to make atomic.

If the event log and the application state are partitioned in the same way (for example, processing an event for a customer in partition 3 only requires updating partition 3 of the application state), then a straightforward single-threaded log consumer needs no concurrency control for writes — by construction, it only processes a single event at a time (see also "Actual serial execution" on page 243). The log removes the nondeterminism of concurrency by defining a serial order of events in a partition [22]. If an event touches multiple state partitions, a bit more work is required, which we will discuss later in this chapter.

### Immutability and deletion

Many systems that don't use an event-sourced model nevertheless rely on immutability: for example, Datomic uses immutable data structures to query snapshots of a database, including historical snapshots from past points in time (see "Indexes and snapshot isolation" on page 232). Version control systems such as git, Mercurial and Fossil also rely on immutable data to preserve version history of files.

Storage is now so cheap that keeping all events forever is a viable option for all but the very largest systems. However, there may still be circumstances where you need data to really be deleted, in spite of all immutability. For example, privacy regulations

may require deleting a user's personal information after they close their account, or an accidental leak of sensitive information may need to be contained.

In these circumstances, it's not sufficient to just append another event to the log to indicate that the prior data should be considered deleted — you actually want to rewrite history and pretend that the data was never written in the first place. For example, Datomic calls this feature *excision* [53], and the Fossil version control system has a similar concept called *shunning* [54].

Truly deleting data is surprisingly hard [55], since copies can live in many places: for example, storage engines, filesystems and SSDs often write to a new location rather than overwriting in-place [47], and backups are often deliberately immutable to prevent accidental deletion or corruption. Deletion is more a matter of "making it harder to retrieve the data" than a matter of "making it impossible to retrieve the data". Nevertheless, it is sometimes required.

## Processing Streams

So far in this chapter we have talked about where streams come from (user activity events, sensors, and writes to databases), and we have talked about how streams are transported (through direct messaging, via message brokers, and event logs).

What remains is to discuss what you can do with the stream once you have it — namely, you can process it. Broadly, there are three options:

1. You can take the data in the events and write it to a database, cache, search index or similar storage system, from where it can then be queried by other clients. As shown in Figure 11-5, this is a good way of keeping a database in sync with changes happening in other parts of the system — especially if the stream consumer is the only client writing to the database. Writing to a storage system is the streaming equivalent of what we discussed in "The output of batch workflows" on page 398.
2. You push the events to users in some way, for example by sending email alerts or push notifications, or by streaming the events to a realtime dashboard where they are visualized. In this case, a human is the ultimate consumer of the stream.
3. You can process one or more input streams to produce one or more output streams. Streams may go through a pipeline consisting of several such processing stages, before they eventually end up at an output (option 1 or 2).

In the rest of this chapter, we will discuss option 3: processing streams to produce other, derived streams. A piece of code that processes streams like this is known as an *operator* or a *job*. It is closely related to the Unix processes and the MapReduce jobs we discussed in Chapter 10, and the pattern of dataflow is similar: a stream processor

consumes input streams in a read-only fashion, and writes its output to a different location in an append-only fashion.

The patterns for partitioning and parallelization in stream processors are also very similar to MapReduce and the dataflow engines we saw in Chapter 10, so we won't repeat that topic here. Basic mapping operations such as transforming and filtering records also work the same.

The one crucial difference to batch jobs is that a stream never ends. This difference has many implications: as discussed at the start of this chapter, sorting does not make sense with an unbounded dataset, and so sort-merge joins (see "Reduce-side joins and grouping" on page 391) do not apply. Fault-tolerance mechanisms must also change: with a batch job that has been running for a few minutes, a failed task can simply be restarted from the beginning — but with a stream job that has been running for several years, restarting from the beginning may not be a viable option.

## Uses of stream processing

Stream processing has long been used for monitoring purposes, where an organization wants to be alerted if certain things happen. For example:

- fraud detection systems need to determine if the usage patterns of a credit card have unexpectedly changed, and block the card if it is likely to have been stolen;
- trading systems need to examine price changes in a financial market and execute trades according to specified rules;
- manufacturing systems need to monitor the status of machines in a factory, and quickly identify the problem if there is a malfunction;
- military and intelligence systems need to track the activities of a potential aggressor, and raise the alarm if there are signs of an attack.

These kinds of application require quite sophisticated pattern-matching and correlations. However, other uses of stream processing have also emerged over time. In this section we will briefly compare and contrast some of these applications.

### Complex event processing

*Complex event processing* (CEP) is an approach developed in the 1990s for analyzing event streams, especially geared towards the kind of application that requires searching for certain event patterns [56, 57]. Similarly to the way that a regular expression allows you to search for certain patterns of characters in a string, CEP allows you to specify rules to search for certain patterns of events in a stream.

CEP systems often use a high-level declarative query language like SQL, or a graphical user interface, to describe the pattern of events that should be detected. These

queries are submitted to a processing engine that consumes the input streams and internally maintains a state machine that performs the required matching. When a match is found, the engine emits a *complex event* (hence the name) with the details of the event pattern that was detected.

In these systems, the relationship between queries and data is reversed compared to normal databases. Usually, a database stores data persistently, and treats queries as transient: when a query comes in, the database searches for data matching the query, and then forgets about the query. CEP engines are the other way round: queries are stored long-term, and events from the input streams continuously flow past the queries in search of a query that matches the event.

Implementations of CEP include Esper [58], IBM InfoSphere Streams [59], Apama, TIBCO StreamBase, and SQLstream.

#### Stream analytics

Another area in which stream processing is used is for *analytics* on streams. The boundary between CEP and stream analytics is blurry, but as a general rule, analytics tends to be less interested in finding specific event sequences, and is more oriented towards aggregations and statistical metrics over a large number of events, for example:

- measuring the rate of some type of event (how often it occurs per time interval),
- calculating the rolling average of a value over some time period, or
- comparing current statistics to previous time intervals (e.g. to detect trends, or to alert on metrics that are unusually high or low compared to the same time last week).

Such statistics are usually computed over fixed time intervals — for example, you might want to know the average number of queries per second to a service over the last five minutes, and their 99th percentile response time during that period. Averaging over a few minutes smoothes out irrelevant fluctuations from one second to the next, while still giving you a timely picture of any changes in traffic pattern. The time interval over which you aggregate is known as a *window*, and we will look into windowing in more detail in "Reasoning about time" on page 452.

Stream analytics systems sometimes use probabilistic algorithms, such as bloom filters (which we encountered in "SSTables and LSM-trees" on page 74) for set membership, HyperLogLog [60] for cardinality estimation, and various percentile estimation algorithms (see "Percentiles in Practice" on page 14). Probabilistic algorithms produce approximate results, but have the advantage of requiring significantly less memory in the stream processor than exact algorithms. This use of approximation algorithms sometimes leads people to believe that stream processing systems are

always lossy and inexact, but that is wrong: there is nothing inherently approximate about stream processing [61].

Many open source distributed stream processing frameworks are designed with analytics in mind: for example, Apache Storm, Spark Streaming, Flink, Concord, Samza, and Kafka Streams [62]. Hosted services include Google Cloud Dataflow and Azure Stream Analytics.

### Maintaining materialized views

We saw in "Databases and streams" on page 436 that a stream of changes to a database can be used to keep derived data systems, such as caches, search indexes and data warehouses, up-to-date with a source database. We can regard these examples as specific cases of maintaining *materialized views* (see "Aggregation: Data cubes and materialized views" on page 98): deriving an alternative view onto some dataset, so that you can query it efficiently, and updating that view whenever the underlying data changes [46].

Similarly, in event sourcing, application state is maintained by applying a log of events; here the application state is also a kind of materialized view. Unlike stream analytics scenarios, it is usually not sufficient to consider only events within some time window: building the materialized view requires *all* events that ever happened — perhaps after log compaction has discarded obsolete events (see "Log compaction" on page 441). In effect, you need a window that stretches all the way back to the beginning of time.

In principle, any stream processor could be used for materialized view maintenance, although the need to maintain events forever runs counter to the assumptions of some analytics-oriented frameworks. Samza and Kafka Streams support this kind of usage, building upon Kafka's support for log compaction [63].

### Search on streams

Besides CEP, which allows searching for patterns consisting of multiple events, there is also sometimes a need to search for individual events based on complex criteria, such as full-text search queries.

For example, media monitoring services subscribe to feeds of news articles and broadcasts from media outlets, and search for any news mentioning companies, products or topics of interest. This is done by formulating a search query in advance, and then continually matching the stream of news items against this query. Similar features exist on some websites: for example, users of real estate websites can ask to be notified when a new property matching their search criteria appears on the market.

Conventional search engines first index the documents and then run queries over the index. By contrast, searching a stream turns the processing on its head: the queries are stored, and the documents run past the queries. In the simplest case, you can test every document against every query, although this can get slow if you have a large number of queries. To optimize the process, it is possible to index the queries as well as the documents, and thus narrow down the set of queries that may match [64].

#### Message passing and RPC

In "Message passing data flow" on page 132 we discussed message-passing systems as an alternative to RPC, i.e. as a mechanism for services to communicate, as used for example in the actor model. Although these systems are also based on messages and events, we normally don't think of them as stream processors:

- Actor frameworks are primarily a mechanism for managing concurrency and distributed execution of communicating modules, whereas stream processing is primarily a data management mechanism.
- Communication between actors is often ephemeral and one-to-one, whereas event logs are durable and multi-subscriber.
- Actors can communicate in arbitrary ways (including cyclic request-response patterns), but stream processors are usually set up in acyclic pipelines where every stream is the output of one particular job, and derived from a well-defined set of input streams.

That said, there is some cross-over area between RPC-like systems and stream processing. For example, Apache Storm has a feature called *distributed RPC*, which allows user queries to be farmed out to a set of nodes that also process event streams; these queries are then interleaved with events from the input streams, and results can be aggregated and sent back to the user [65]. It is also possible to build stream processors on top of actor frameworks, although it is worth examining the fault tolerance guarantees that this approach can provide.

## Reasoning about time

Stream processors often need to deal with time, especially when used for analytics purposes, which often use time windows such as "last five minutes". It might seem that the meaning of "last five minutes" should be unambiguous and clear, but unfortunately the notion is surprisingly tricky.

In a batch process, the processing tasks rapidly crunch through a large collection of historical events. If some kind of breakdown by time needs to happen, the batch process needs to look at the timestamp embedded in each event. There is no point in looking at the system clock of the machines running the batch process, because the

time at which the process is run has nothing to do with the time at which the events actually occurred.

A batch process may read a year worth of historical events within a few minutes; in most cases, the timeline of interest is the year of history, not the few minutes of processing. Moreover, using the timestamp in the events allows the processing to be deterministic: running the same process again on the same input yields the same result (see "Fault tolerance" on page 410).

On the other hand, many stream processing frameworks use the local system clock on the processing machine (the *processing time*) to determine windowing [66]. This approach has the advantage of being simple, and it is reasonable if the delay between event creation and event processing is negligibly short. However, it breaks down if there is any significant processing lag, i.e. if the processing may happen noticeably later than the time at which the event actually occurred.

#### Event time versus processing time

There are many reasons why processing may be delayed: network faults (see "Unreliable Networks" on page 269), a performance issue leading to contention in the message broker or processor, a restart of the stream consumer, or re-processing past events (see "Replaying old messages" on page 436) while recovering from a fault, or after fixing a bug in the code.

Moreover, message delays also lead to unpredictable ordering of messages. For example, say a user first makes one web request (which is handled by web server A), and then a second request (which is handled by server B). A and B emit events describing the requests they handled, but B's event reaches the message broker before A's event does. Now stream processors will first see the B event and then the A event, even though they actually occurred in the opposite order.

If it helps to have an analogy, consider the Star Wars movies: Episode IV was released in 1977, Episode V in 1980, and Episode VI in 1983, followed by episodes I, II and III in 1999, 2002 and 2005 respectively, and Episode VII in 2015 [67]. If you watched the movies in the order they came out, the order in which you processed the movies is inconsistent with the order of their narrative. (The episode number is like the event timestamp, and the date when you watched the movie is the processing time.) As humans, we are able to cope with such discontinuities, but stream processing algorithms need to be specifically written to accommodate such timing and ordering issues.

Confusing event time and processing time leads to bad data. For example, say you have a stream processor that measures the rate of requests (counting the number of requests per second). If you redeploy the stream processor, it may be shut down for a minute, and process the backlog of events when it comes back up. If you measure the rate based on the processing time, it will look as if there was a sudden anomalous

spike of requests while processing the backlog, when in fact the real rate of requests was steady (Figure 11-7).

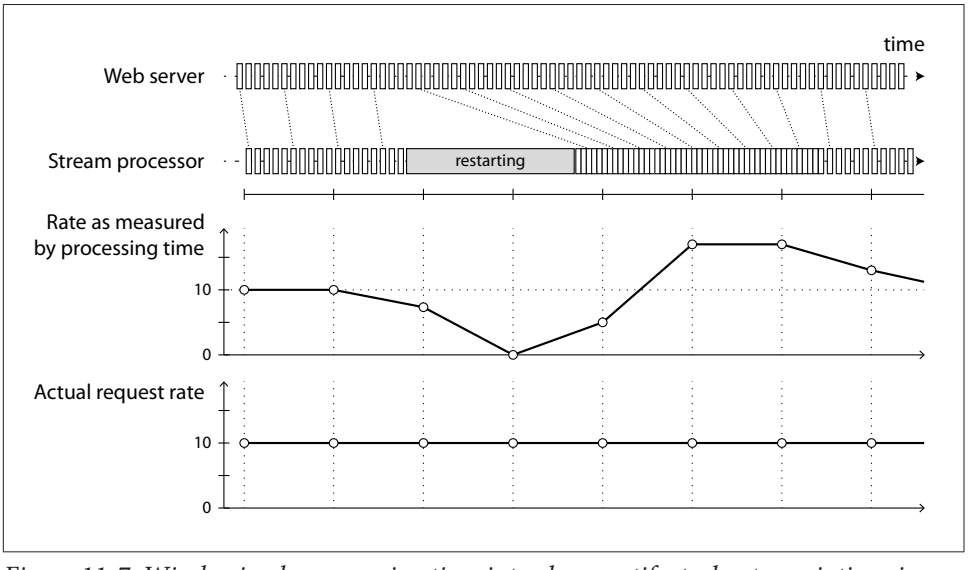


*Figure 11-7. Windowing by processing time introduces artifacts due to variations in processing rate.*

### Knowing when you're ready

A tricky problem when defining windows in terms of event time is that you can never be sure when you have received all of the events for a particular window, or whether there are some events still to come.

For example, say you're grouping events into one-minute windows, so that you can count the number of requests per minute. You have counted some number of events with timestamps that fall in the 37th minute of the hour, and then time has moved on, and now most of the incoming events fall within the 38th and 39th minute of the hour. When do you declare that you have finished the window for the 37th minute, and output its counter value?

In general, it's impossible to be sure that you have received all the events for a window. You can time out and declare a window ready after you have not seen any new events for a while, but it could still happen that some events were buffered on another machine somewhere, which couldn't yet be sent due to a network interruption, but which will turn up later.

If you are using windows based on event timestamps, you need to be able to handle such *straggler* events that arrive after the window has already been declared complete. Broadly, you have two options [1]:

1. Ignore the straggler events, as they are probably a small percentage of events in normal circumstances. You can track the number of dropped events as a metric, and alert if you start dropping a significant amount of data.
2. Recalculate the value for the window with the stragglers included, and issue a *correction* by publishing the updated value (possibly retracting the previous output first).

In some cases it is possible to use a *low watermark* to indicate things like "from now on there will be no more messages with a timestamp earlier than *t*", and to have consumers wait for the watermark as an indication that the window is ready [68]. However, if timestamps are generated by clients, you cannot be sure whether there are still any pending events somewhere in the system, so stragglers (with a timestamp older than the low watermark) are still possible.

#### Whose clock are you using, anyway?

Assigning timestamps to events is even more difficult when events can be buffered at several points in the system. For example, consider a mobile app that reports events for usage metrics to a server. The app may be used while offline, in which case it will buffer events locally on the device, and send them to a server when an internet connection is next available (hours or days later). To any consumers of this stream, the events will appear as extremely delayed stragglers.

In this context, the timestamp on the events should really be the time at which the user interaction occurred, according to the mobile device's local clock. However, the clock on a user-controlled device often cannot be trusted, as it may be accidentally or deliberately set to the wrong time (see "Clock synchronization and accuracy" on page 281). The time at which the event was received by the server (according to the server's clock) is more likely to be accurate, but less meaningful in terms of describing the user interaction.

To adjust for incorrect device clocks, one approach is to log three timestamps [69]:

- the time at which the event occurred, according to the device clock;
- the time at which the event was sent to the server, according to the device clock;
- the time at which the event was received by the server, according to the server clock.

This allows you to estimate the offset between the device clock and the server clock (assuming the network delay is negligible compared to the required timestamp accuracy), and thus estimate the true time at which the event actually occurred (assuming the device clock offset does not change between the time the event occurred and the time it was sent to the server).

This problem is not unique to stream processing — batch processing suffers from exactly the same issue. It is just more noticeable in a streaming context, where we are more aware of the passage of time.

### Types of window

Once you know how the timestamp of an event should be determined, the next step is to decide how windows over time periods should be defined. The window can then be used for aggregations, for example to count events, or to calculate the average of values within the window. Several types of window are in common use [66, 70]:

*Tumbling window*
: A tumbling window has a fixed length, and every event belongs to exactly one window. For example, if you have a 1-minute tumbling window, all the events with timestamps between 10:03:00 and 10:03:59 are grouped into one window, events between 10:04:00 and 10:04:59 into the next window, and so on. You could implement a 1-minute tumbling window by taking each event timestamp and rounding it to the nearest minute.

*Hopping window*
: A hopping window also has a fixed length, but allows windows to overlap in order to provide some smoothing. For example, a 5-minute window with a hop size of 1 minute would contain the events between 10:03:00 and 10:07:59, then the next window would cover events between 10:04:00 and 10:08:59, and so on. You can implement this hopping window by first calculating 1-minute tumbling windows, and then aggregating over several adjacent windows.

*Sliding window*
: A sliding window contains all the events that occur within some interval of each other. For example, a 5-minute sliding window would cover events at 10:03:39 and 10:08:12, because they are less than 5 minutes apart (note that tumbling and hopping 5-minute windows would not have put these two events in the same window, as they use fixed boundaries). A sliding window can be implemented by keeping a buffer of events sorted by time, and removing old events when they expire from the window.

*Session window*
: Unlike the other window types, a session window has no fixed duration. Instead, it is defined by grouping together all events for the same user that occur closely together in time, and the window ends when the user has been inactive for some time (for example, if there have been no events for 30 minutes). Sessionization is a common requirement for website analytics (see "GROUP BY" on page 394).

## Stream joins

In Chapter 10 we discussed how batch jobs can join datasets by key, and how such joins are an important part of data pipelines. Since stream processing generalizes data pipelines to incremental processing of unbounded datasets, there is exactly the same need for joins on streams.

However, the fact that new events can appear anytime on a stream makes joins on streams more challenging than in batch jobs. To understand the situation better, let's distinguish three different types of join: *stream-stream* joins, *stream-table* joins, and *table-table* joins [71]. In the following sections we'll illustrate each by example.

### Stream-stream join (window join)

Say you have a search feature on your website, and you are working on improving the ranking of search results. Every time someone types a search query, you log an event containing the query and the results returned. Every time someone clicks one of the search results, you log another event recording the click. In order to calculate the click-through rate for each search result, you need to bring together the events for the search action and the click action, which are connected by having the same session ID [72].

The click may never come if the user abandons their search, and even if it comes, the time between the search and the click may be highly variable: in many cases it might be a few seconds, but it could be as long as days or weeks (if a user runs a search, forgets about that browser tab, and then returns to the tab and clicks a result sometime later). Due to variable network delays, the click event may even arrive before the search event. You can choose a suitable window for the join — for example, you may choose to join a click with a search if they occur at most one hour apart.

Note that embedding the details of the search in the click event is not equivalent to joining the events: that would only tell you about the cases where the user clicked a search result, but not about the searches where the user did not click any of the results. In order to measure search quality you need accurate click-through rates, for which you need both the search events and the click events.

To implement this type of join, a stream processor needs to maintain *state*: for example, all the events that occurred in the last hour, indexed by session ID. Whenever a search event or click event occurs, it is added to the appropriate index, and it also checks the other index to see if another event for the same session ID has already arrived.

### Stream-table join (stream enrichment)

In "Example: analysis of user activity events" on page 392 (Figure 10-2) we saw an example of a batch job joining two datasets: a set of user activity events and a data-

base of user profiles. It is natural to think of the user activity events as a stream, and to perform the same join on a continuous basis in a stream processor: the input is a stream of activity events containing a user ID, and the output is a stream of *enriched* activity events in which the user ID has been augmented with profile information about the user.

To perform this join, the stream process needs to look at one activity event at a time, look up the event's user ID in the database, and add the profile information to the activity event. The database lookup could be implemented by querying a remote database; however, as discussed in "Example: analysis of user activity events" on page 392, this is likely to be slow, and risks overloading the remote database [63].

Another approach is to load a copy of the database into the stream processor, so that it can be queried locally without a network round-trip. This is very similar to the hash joins we discussed in "Map-side joins" on page 396: the local copy of the database might be an in-memory hash table if it is small enough, or an index on local disk.

The difference to batch jobs is that a batch job uses a point-in-time snapshot of the database as input, whereas a stream processor is long-running, and the contents of the database is likely to change over time, so the local copy of the data needs to be kept up-to-date. This issue can be solved by change data capture: the stream processor can subscribe to a changelog of the profiles database as well as the stream of activity events. When a profile is created or modified, the stream processor updates its local copy.

#### Table-table join (materialized view maintenance)

Consider the Twitter timeline example that we discussed in "Describing load" on page 9. We said that when a user wants to view their home timeline, it is too expensive to iterate over all the people the user is following, find their recent tweets, and merge them.

Instead, we want a timeline cache: a kind of per-user "inbox" to which tweets are written as they are sent, so that reading the timeline is a single lookup. Materializing and maintaining this cache requires the following event processing:

- When user $u$ sends a new tweet, it is added to the timeline of every user who is following $u$.
- When a user deletes a tweet, it is removed from all users' timelines.
- When user $u_1$ starts following user $u_2$, recent tweets by $u_2$ are added to $u_1$'s timeline.
- When user $u_1$ unfollows user $u_2$, tweets by $u_2$ are removed from $u_1$'s timeline.

To implement this cache maintenance in a stream processor, you need streams of events for tweets (sending and deleting) and for follow relationships (following and unfollowing). The stream process needs to maintain a database containing the set of followers for each user, so that it knows which timelines need to be updated when a new tweet arrives [73].

Another way of looking at this stream process is that it maintains a materialized view for a query that joins two tables (tweets and follows), something like the following:

```
SELECT follows.follower_id AS timeline_id,
  array_agg(tweets.* ORDER BY tweets.timestamp DESC)
FROM tweets
JOIN follows ON follows.followee_id = tweets.sender_id
GROUP BY follows.follower_id
```

The join of the streams corresponds directly to the join of the tables in that query. The timelines are effectively a cache of the result of this query, updated every time the underlying tables change.[ii]

### Time-dependence of joins

The three types of join above (stream-stream, stream-table and table-table) have a lot in common: they all require the stream processor to maintain some state (search and click events, user profiles, or follower list) based on one join input, and query that state on messages from the other join input.

The order of the events that maintain the state is important (it matters whether you first follow and then unfollow, or the other way round). In a partitioned log, the ordering of events within a single partition is preserved, but there is typically no ordering guarantee across different streams or partitions.

This raises a question: if events on different streams happen around a similar time, in which order are they processed? In the stream-table join example, if a user updates their profile, which activity events are joined with the old profile (processed before the profile update), and which are joined with the new profile (processed after the profile update)? Put another way: if state changes over time, and you join with some state, what point in time do you use for the join [41]?

If the ordering of events across streams is undetermined, the join becomes non-deterministic, which means you cannot re-run the same job on the same input and necessarily get the same result: the events on the input streams may be interleaved in a different way when you run the job again. It is possible for a stream processor to log

ii. If you regard a stream as the derivative of a table, as in Figure 11-6, and regard a join as a product of two tables $u \cdot v$, something interesting happens: the stream of changes to the materialized join follows the product rule $(u \cdot v)' = u'v + uv'$. In words: whenever the tweets change, it is joined with the current follows, and whenever the follows change, it is joined with the current tweets [45, 46].

the interleaving of messages [74], but this alone is not enough when recovering from faults, as we shall see in the next section.

## Fault tolerance

In the final section of this chapter, let's consider how stream processors can tolerate faults. We saw in Chapter 10 that batch processing frameworks can tolerate faults fairly easily: if a task in a MapReduce job fails, it can simply be started again on another machine, and the output of the failed task is discarded. This is possible because input files are immutable, each task writes its output to a separate file on HDFS, and output is only made visible when a task completes successfully.

In particular, the batch approach to fault tolerance ensures that the output of the batch job is the same as if nothing had gone wrong, even if in fact some tasks did fail. It appears as though every input record was processed exactly once — no records are skipped, and none are processed twice. Although restarting tasks means that records may in fact be processed multiple times, the *visible effect* in the output is as if they had only been processed once, a principle known as *exactly-once semantics*.

The same issue of fault tolerance arises in stream processing, but it is less straightforward to handle: waiting until a task is finished before making its output visible is not an option, because a stream is infinite and so you can never finish processing it.

### Microbatching and checkpointing

One solution is to break the stream into small blocks, and treat each block like a miniature batch process. This approach is called *microbatching*, and it is used in Spark Streaming [75]. The batch size is typically around 1 second, which is a performance compromise: smaller batches incur greater scheduling and coordination overhead, while larger batches mean a longer delay before results of the stream processor become visible.

Microbatching also implicitly provides a tumbling window equal to the batch size (windowed by processing time, not event timestamps); any jobs that require larger windows need to explicitly carry over state from one microbatch to the next.

A variant approach, used in Apache Flink, periodically generates rolling checkpoints of state and writes them to durable storage [76, 77]. If a stream operator crashes, it can restart from its most recent checkpoint, and discard any output generated between the last checkpoint and the crash. The checkpoints are triggered by barriers in the message stream, similar to the boundaries between microbatches, but without forcing a particular window size.

Within the confines of the stream processing framework, the microbatching and checkpointing approaches provide the same exactly-once semantics as batch processing. However, as soon as output leaves the stream processor (for example, by writing

to a database, sending messages to an external message broker, or sending emails), the framework is no longer able to discard the output of a failed batch.

In this case, restarting a failed task causes the external side-effect to happen twice, and microbatching or checkpointing alone is not sufficient to prevent this problem.

#### Atomic commit revisited

In order to give the appearance of exactly-once processing in the presence of faults, we need to ensure that all outputs and side-effects of processing an event take effect *if and only if* the processing is successful. Those effects include any messages sent to downstream operators or external messaging systems (including email or push notifications), any database writes, any changes to operator state, and any acknowledgement of input messages (including moving the consumer offset forward in a log-based message broker).

Those things either all need to happen atomically, or none of them must happen, but they should not go out of sync with each other. If this approach sounds familiar, it is because we discussed it in "Exactly-once message processing" on page 351 in the context of distributed transactions and 2-phase commit.

In Chapter 9 we discussed the problems in the traditional implementation of distributed transactions, such as XA. However, in more restricted environments it is possible to implement such an atomic commit facility efficiently. This approach is used in Google Cloud Dataflow [68, 76], and there are plans to add similar features to Apache Kafka [78]. The approach relies on writing the transaction commit as a single object to a fault-tolerant datastore, since a single-object write can be made atomic fairly easily (see "Single-object writes" on page 221).

#### Idempotence

Our goal is to discard the partial output of any failed tasks, so that they can be safely retried without taking effect twice. Distributed transactions are one way of achieving that goal, but another way is to rely on *idempotence* [79].

An idempotent operation is one that you can perform multiple times, and it has the same effect as if you performed it only once. For example, setting a key in a key-value store to some fixed value is idempotent (writing the value again simply overwrites the value with an identical value), whereas incrementing a counter is not idempotent (performing the increment again means the value is incremented twice).

Even if an operation is not naturally idempotent, it can often be made idempotent with a bit of extra metadata. For example, when consuming messages from Kafka, every message has a persistent, monotonically increasing offset. When writing a value to an external database, you can include the offset of the message that triggered the

last write with the value. Thus, you can tell whether an update has already been applied, and avoid performing the same update again.

The state handling in Storm's Trident is based on a similar idea [65]. Relying on idempotence implies several assumptions: restarting a failed task must replay the same messages in the same order (a log-based message broker does this), the processing must be deterministic, and no other node may concurrently update the same value [80].

When failing over from one processing node to another, fencing may be required (see "The leader and the lock" on page 293) to prevent interference from a node that is thought to be dead but is actually alive. Despite all those caveats, idempotent operations can be an effective way of achieving exactly-once semantics with only a small overhead.

#### Rebuilding state after a failure

Any stream process that requires state — for example, any windowed aggregations (such as counters, averages and histograms) and any tables and indexes used for joins — must ensure that this state can be recovered after a failure.

One option is to keep the state in a remote datastore and replicate it, although (as discussed in "Stream-table join (stream enrichment)" on page 457) having to query a remote database for each individual message can be slow. A better option is to keep state local to the stream processor, and to replicate it periodically. Then, when the stream processor is recovering from a failure, the new task can read the replicated state, and resume processing without data loss.

For example, Flink periodically captures snapshots of operator state and writes them to durable storage such as HDFS [76, 77]; Samza and Kafka Streams replicate state changes by sending them to a dedicated Kafka topic with log compaction, similar to change data capture [71, 81].

In some cases, it may not even be necessary to replicate the state, because it can be rebuilt from the input streams. For example, if the state consists of aggregations over a fairly short window, it may be fast enough to simply replay the input events corresponding to that window. If the state is a local replica of a database, maintained by change data capture, the database can also be rebuilt from the log-compacted change stream (see "Log compaction" on page 441).

## Summary

In this chapter we have discussed event streams, what purposes they serve and how to process them. In some ways, stream processing is very much like the batch processing we discussed in Chapter 10, but done continuously on an unbounded (never-ending)

stream rather than on a fixed-size input. From this perspective, message brokers and event logs serve as the streaming equivalent of a filesystem.

We spent some time comparing two types of message broker:

*AMQP/JMS-style message brokers*
: The broker assigns individual messages to consumers, and consumers acknowledge individual messages when they have been successfully processed. Messages are deleted from the broker once they have been acknowledged. This approach is appropriate as an asynchronous form of RPC (see also "Message passing data flow" on page 132), for example in a task queue, where the exact order of message processing is not important, and where there is no need to go back and read old messages again after they have been processed.

*Log-based message brokers*
: The broker assigns all messages in a partition to the same consumer node, and always delivers messages in the same order. Parallelism is achieved through partitioning, and consumers track their progress by checkpointing the offset of the last message they have processed. The broker retains messages on disk, so it is possible to jump back and re-read old messages if necessary.

The log-based approach has similarities to replication logs found in databases (see Chapter 5) and log-structured storage engines (see Chapter 3). We saw that this approach is especially appropriate for stream processing systems that consume input streams and generate derived state or derived output streams.

In terms of where streams come from, we discussed several possibilities: user activity events, sensors providing periodic readings, and data feeds (e.g. market data in finance) are naturally represented as streams. We saw that it can also be useful to think of the writes to a database as a stream: it can capture the changelog, i.e. the history of all changes made to a database, either implicitly through change data capture or explicitly through event sourcing. Log compaction allows the stream to retain a full copy of the contents of a database.

Representing databases as streams opens up powerful opportunities for integrating systems. You can keep derived data systems such as search indexes, caches and analytics systems continually up-to-date by consuming the log of changes and applying them to the derived system. You can even build fresh views onto existing data by starting from scratch and consuming the log of changes from the beginning all the way to the present.

The facilities for maintaining state as streams and replaying messages are also the basis for the techniques that enable streaming joins and fault tolerance in various stream processing frameworks. We discussed several purposes of stream processing, including searching for event patterns (complex event processing), computing win-

dowed aggregations (stream analytics), and keeping derived data systems up-to-date (materialized views).

We then discussed the difficulties of reasoning about time in a stream processor, including the distinction between processing time and event timestamps, and the problem of dealing with straggler events that arrive after you thought your window was complete.

We distinguished three types of join that may appear in stream processes:

*Stream-stream joins*
: Matching two events that occur within some window of time — for example, two actions taken by the same user within 30 minutes of each other.

*Stream-table joins*
: One input stream consists of activity events, while the other is a database changelog. The changelog keeps a local copy of the database up-to-date, while activity events query the database and output an enriched activity event.

*Table-table joins*
: Both input streams are database changelogs. In this case, every change on one side is joined with the latest state of the other side. The result is a stream of changes to the materialized view of the two tables.

Finally, we discussed techniques for achieving fault tolerance and exactly-once semantics in a stream processor. As with batch processing, we need to discard the partial output of any failed tasks. However, since a stream process is long-running and produces output continuously, we can't simply discard all output. Instead, a finer-grained recovery mechanism can be used, based on microbatching, checkpointing, transactions, or idempotent writes.

## References

[1] Tyler Akidau, Robert Bradshaw, Craig Chambers, et al.: "The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing," *Proceedings of the VLDB Endowment*, volume 8, number 12, pages 1792–1803, August 2015. doi:10.14778/2824032.2824076

[2] Harold Abelson, Gerald Jay Sussman, and Julie Sussman: *Structure and Interpretation of Computer Programs*, Second Edition. MIT Press, July 1996. ISBN: 9780262510875, online at mitpress.mit.edu.

[3] Joseph M Hellerstein and Michael Stonebraker: *Readings in Database Systems*, Fourth Edition. MIT Press, January 2005. ISBN: 978-0-262-69314-1, redbook.cs.berkeley.edu.

[4] Don Carney, Uğur Çetintemel, Mitch Cherniack, et al.: "Monitoring Streams – A New Class of Data Management Applications," at *28th International Conference on Very Large Data Bases* (VLDB), August 2002.

[5] Vicent Martí: "Brubeck, a statsd-compatible metrics aggregator," githubengineering.com, 15 June 2015.

[6] Seth Lowenberger: "MoldUDP64 Protocol Specification V 1.00," nasdaqtrader.com, July 2009.

[7] Pieter Hintjens: *ZeroMQ – The Guide*. O'Reilly Media, March 2013. ISBN: 978-1-4493-3404-8, online at zguide.zeromq.org.

[8] Ian Malpass: "Measure Anything, Measure Everything," codeascraft.com, 15 February 2011.

[9] Dieter Plaetinck: "25 graphite, grafana and statsd gotchas," blog.raintank.io, 3 March 2016.

[10] Jeff Lindsay: "Web hooks to revolutionize the web," progrium.com, 3 May 2007.

[11] Jim N Gray: "Queues Are Databases," Microsoft Research Technical Report MSR-TR-95-56, December 1995.

[12] Matthew Sackman: "Pushing Back," lshift.net, 5 May 2016.

[13] Mark Hapner, Rich Burridge, Rahul Sharma, et al.: "JSR-343 Java Message Service (JMS) 2.0 Specification," jms-spec.java.net, March 2013.

[14] Sanjay Aiyagari, Matthew Arrott, Mark Atwell, et al.: "AMQP: Advanced Message Queuing Protocol Specification," Version 0-9-1, November 2008.

[15] "Apache Kafka 0.9 documentation," kafka.apache.org, November 2015.

[16] Jay Kreps, Neha Narkhede, and Jun Rao: "Kafka: a Distributed Messaging System for Log Processing," at *6th International Workshop on Networking Meets Databases* (NetDB), June 2011.

[17] "Amazon Kinesis Streams Developer Guide," docs.aws.amazon.com, April 2016.

[18] Leigh Stewart and Sijie Guo: "Building DistributedLog: Twitter's high-performance replicated log service," blog.twitter.com, 16 September 2015.

[19] "DistributedLog documentation," Twitter Inc., distributedlog.io, May 2016.

[20] Jay Kreps: "Benchmarking Apache Kafka: 2 Million Writes Per Second (On Three Cheap Machines)," engineering.linkedin.com, 27 April 2014.

[21] Kartik Paramasivam: "How We're Improving and Advancing Kafka at LinkedIn," engineering.linkedin.com, 2 September 2015.

[22] Jay Kreps: "The Log: What every software engineer should know about real-time data's unifying abstraction," engineering.linkedin.com, 16 December 2013.

[23] Shirshanka Das, Chavdar Botev, Kapil Surlaker, et al.: "All Aboard the Databus!," at *3rd ACM Symposium on Cloud Computing* (SoCC), October 2012.

[24] Yogeshwer Sharma, Philippe Ajoux, Petchean Ang, et al.: "Wormhole: Reliable Pub-Sub to Support Geo-replicated Internet Services," at *12th USENIX Symposium on Networked Systems Design and Implementation* (NSDI), May 2015.

[25] P P S Narayan: "Sherpa update," developer.yahoo.com, 8 June 2010.

[26] Martin Kleppmann: "Bottled Water: Real-time integration of PostgreSQL and Kafka," martin.kleppmann.com, 23 April 2015.

[27] Ben Osheroff: "Introducing Maxwell, a mysql-to-kafka binlog processor," developer.zendesk.com, 20 August 2015.

[28] Randall Hauch: "Debezium 0.2.1 Released," debezium.io, 10 June 2016.

[29] "Mongoriver," Stripe, Inc., github.com, September 2014.

[30] Dan Harvey: "Change Data Capture with Mongo + Kafka," at *Hadoop Users Group UK*, August 2015.

[31] "Oracle GoldenGate 12c: Real-time access to real-time information," Oracle White Paper, March 2015.

[32] "Oracle GoldenGate Fundamentals: How Oracle GoldenGate Works," Oracle Corporation, youtube.com, November 2012.

[33] Slava Akhmechet: "Advancing the realtime web," rethinkdb.com, 27 January 2015.

[34] "Firebase Realtime Database Documentation," Google Inc., firebase.google.com, May 2016.

[35] "Apache CouchDB 1.6 Documentation," docs.couchdb.org, 2014.

[36] Matt DeBergalis: "Meteor 0.7.0: Scalable database queries using MongoDB oplog instead of poll-and-diff," info.meteor.com, 17 December 2013.

[37] Neha Narkhede: "Announcing Kafka Connect: Building large-scale low-latency data pipelines," confluent.io, 18 February 2016.

[38] Greg Young: "CQRS and Event Sourcing," at *Code on the Beach*, August 2014.

[39] Martin Fowler: "Event Sourcing," martinfowler.com, 12 December 2005.

[40] Vaughn Vernon: *Implementing Domain-Driven Design*. Addison-Wesley Professional, February 2013. ISBN: 0321834577

[41] H V Jagadish, Inderpal Singh Mumick, and Abraham Silberschatz: "View Maintenance Issues for the Chronicle Data Model," at *14th ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems* (PODS), pages 113–124, May 1995. doi:10.1145/212433.220201

[42] "Event Store 3.5.0 Documentation," Event Store LLP, docs.geteventstore.com, February 2016.

[43] Martin Kleppmann: *Making Sense of Stream Processing*. O'Reilly Media, May 2016.

[44] Sander Mak: "Event-sourced architectures with Akka," at *JavaOne*, September 2014.

[45] Julian Hyde: personal communication, June 2016.

[46] Ashish Gupta and Inderpal Singh Mumick: *Materialized Views: Techniques, Implementations, and Applications*. MIT Press, May 1999. ISBN: 9780262571227

[47] Pat Helland: "Immutability Changes Everything," at *7th Biennial Conference on Innovative Data Systems Research* (CIDR), January 2015.

[48] Martin Kleppmann: "Accounting for Computer Scientists," martin.kleppmann.com, 7 March 2011.

[49] Pat Helland: "Accountants Don't Use Erasers," blogs.msdn.com, 14 June 2007.

[50] Kartik Paramasivam: "Stream Processing Hard Problems – Part 1: Killing Lambda," engineering.linkedin.com, 27 June 2016.

[51] Martin Fowler: "CQRS," martinfowler.com, 14 July 2011.

[52] Greg Young: "CQRS Documents," cqrs.files.wordpress.com, November 2010.

[53] "Datomic Development Resources: Excision," Cognitect, Inc, docs.datomic.com.

[54] "Fossil Documentation: Deleting Content From Fossil," fossil-scm.org, 2016.

[55] Jay Kreps: "The irony of distributed systems is that data loss is really easy but deleting data is surprisingly hard." twitter.com, 30 March 2015.

[56] David C Luckham: "What's the Difference Between ESP and CEP?," complexevents.com, 1 August 2006.

[57] Srinath Perera: "How is stream processing and complex event processing (CEP) different?," quora.com, 3 December 2015.

[58] "Esper Reference, Version 5.4.0," EsperTech Inc., espertech.com, April 2016.

[59] Zubair Nabi, Eric Bouillet, Andrew Bainbridge, and Chris Thomas: "Of Streams and Storms," IBM technical report, developer.ibm.com, April 2014.

[60] Philippe Flajolet, Éric Fusy, Olivier Gandouet, and Frédéric Meunier: "HyperLogLog: the analysis of a near-optimal cardinality estimation algorithm," at *Conference on Analysis of Algorithms* (AofA), pages 137–156, June 2007.

[61] Jay Kreps: "Questioning the Lambda Architecture," oreilly.com, 2 July 2014.

[62] Ian Hellström: "An Overview of Apache Streaming Technologies," baselineblog.wordpress.com, 12 March 2016.

[63] Jay Kreps: "Why local state is a fundamental primitive in stream processing," oreilly.com, 31 July 2014.

[64] Alan Woodward and Martin Kleppmann: "Real-time full-text search with Luwak and Samza," martin.kleppmann.com, 13 April 2015.

[65] "Apache Storm 1.0.1 Documentation," storm.apache.org, May 2016.

[66] Tyler Akidau: "The world beyond batch: Streaming 102," oreilly.com, 20 January 2016.

[67] Stephan Ewen: "Streaming Analytics with Apache Flink," at *Kafka Summit*, April 2016.

[68] Tyler Akidau, Alex Balikov, Kaya Bekiroğlu, et al.: "MillWheel: Fault-Tolerant Stream Processing at Internet Scale," at *39th International Conference on Very Large Data Bases* (VLDB), pages 734–746, August 2013.

[69] Alex Dean: "Improving Snowplow's understanding of time," snowplowanalytics.com, 15 September 2015.

[70] "Windowing (Azure Stream Analytics)," Microsoft Azure Reference, msdn.microsoft.com, April 2016.

[71] "State Management," Apache Samza 0.10 Documentation, samza.apache.org, December 2015.

[72] Rajagopal Ananthanarayanan, Venkatesh Basker, Sumit Das, et al.: "Photon: Fault-tolerant and Scalable Joining of Continuous Data Streams," at *ACM International Conference on Management of Data* (SIGMOD), June 2013. doi:10.1145/2463676.2465272

[73] Martin Kleppmann: "Samza newsfeed demo," github.com, September 2014.

[74] Ben Kirwin: "Doing the Impossible: Exactly-once Messaging Patterns in Kafka," ben.kirw.in, 28 November 2014.

[75] Matei Zaharia, Tathagata Das, Haoyuan Li, Scott Shenker, and Ion Stoica: "Discretized Streams: An Efficient and Fault-Tolerant Model for Stream Processing on Large Clusters," at *4th USENIX Conference in Hot Topics in Cloud Computing* (HotCloud), June 2012.

[76] Kostas Tzoumas, Stephan Ewen, and Robert Metzger: "High-throughput, low-latency, and exactly-once stream processing with Apache Flink," data-artisans.com, 5 August 2015.

[77] Paris Carbone, Gyula Fóra, Stephan Ewen, Seif Haridi, and Kostas Tzoumas: "Lightweight Asynchronous Snapshots for Distributed Dataflows," *arXiv:1506.08603 [cs.DC]*, 29 June 2015.

[78] Flavio Junqueira: "Making sense of exactly-once semantics," at *Strata+Hadoop World London*, June 2016.

[79] Pat Helland: "Idempotence is not a medical condition," *Communications of the ACM*, volume 55, number 5, page 56, May 2012. doi:10.1145/2160718.2160734

[80] Jay Kreps: "Re: Trying to achieve deterministic behavior on recovery/rewind," email to samza-dev mailing list, 9 September 2014.

[81] Adam Warski: "Kafka Streams - how does it fit the stream processing landscape?," softwaremill.com, 1 June 2016.